

# Survival Model Evaluation: `chicago_licences`

*Fitted with the survival-analysis-pipeline; methodology validated separately against synthetic ground truth*

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| Source data       | <code>datasets/chicago_licences.csv.gz</code>                                            |
| Rows              | 239,721 (82.2% with the ending observed, 17.8% censored)                                 |
| Start dates       | 2002-01-02 to 2026-07-31                                                                 |
| Evaluation        | 5 expanding-window temporal folds                                                        |
| Source of figures | <code>reports/chicago_demo/metrics.json</code> , regenerated by the command in section 8 |

---

## 1. Summary

---

This report evaluates two survival models fitted to `chicago_licences.csv.gz`. It holds 239,721 rows, each observed from its start date. 82.2% have observed endings and 17.8% are censored, still running when observation stopped. Median observed duration is 904 days. The models predict, from what was on file at the start date, how long each row survives.

The like-for-like comparison is the fold mean. Each of the 5 temporal folds fits both models on its own window, and the fold scores average. On concordance, the share of pairs a ranking orders correctly where 0.500 is a coin flip, XGBoost AFT scores 0.585 and the Cox proportional hazards baseline 0.592; the Cox baseline scores higher.

Pooled over 143,833 out-of-time test rows, the AFT model scores 0.573 (95% bootstrap interval 0.571 to 0.575); section 4 explains how the two figures differ.

Both fitted models are saved; the bundle records the recommended model for scoring new rows.

Most of the pooled figure reflects a row's `license_description` group: group means alone score 0.613, within-group comparisons 0.510; section 4 gives the decomposition.

This dataset has no known generating process, so no oracle ceiling bounds these scores, and feature attributions cannot be checked against a true mechanism.

## 2. Data

---

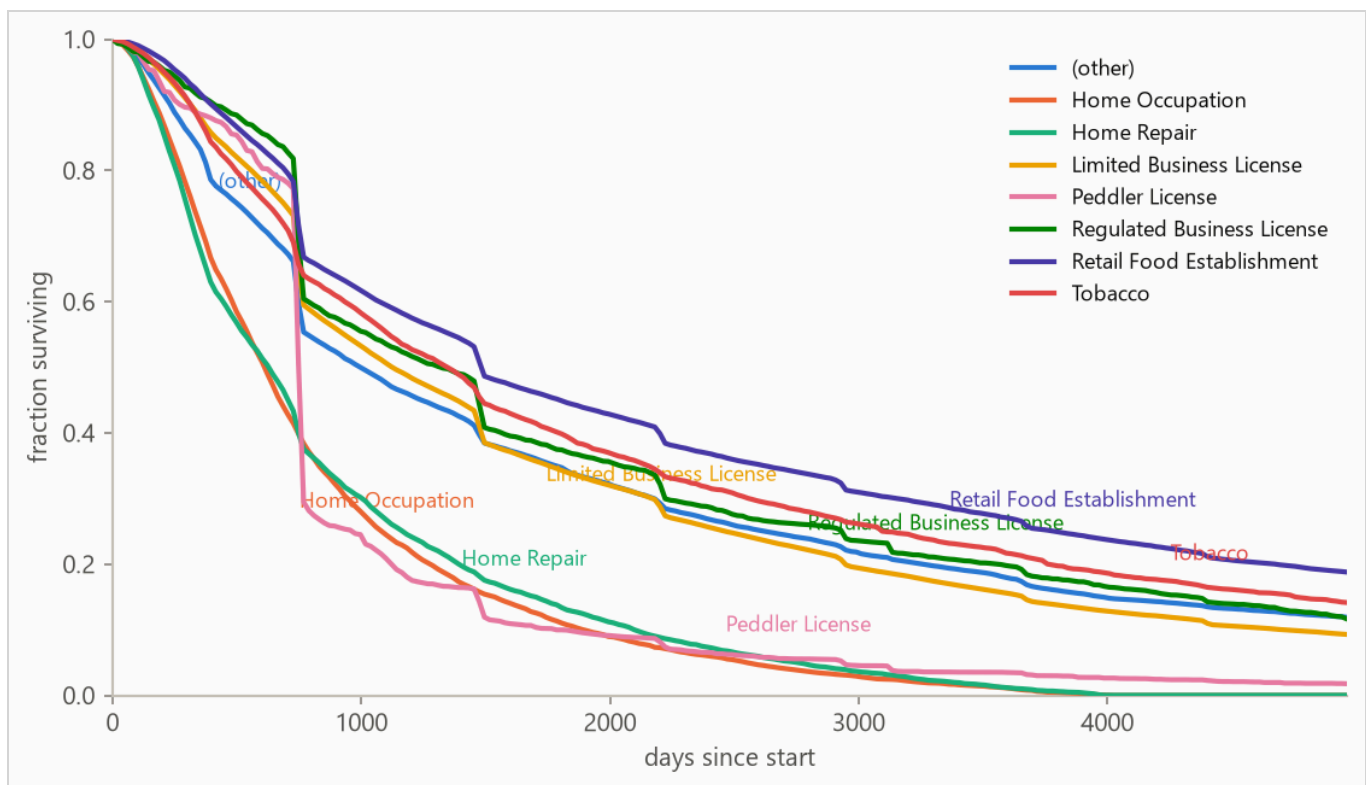
Start dates run 2002-01-02 to 2026-07-31; durations are measured in days. These columns hold numeric codes rather than quantities and were forced to categorical: ward, community\_area, police\_district, zip\_code.

The pipeline treats every row as observed from its own start date. It has no support for left truncation, rows whose life began before the source window, which must be excluded during preparation; whether they were is recorded with the dataset.

For this dataset the preparation step did perform that exclusion. Licences already running when the portal's history begins enter at their first recorded renewal rather than their true start; 79,690 of them, 23 percent of the pull, were removed by keeping only licences whose earliest transaction is a genuine issue. The tell was a spike of 61,351 supposed starts dated 2002, against roughly 14,000 genuine first issues in a normal year. The rule and the counts are recorded in `scripts/run_prepare_chicago.py`.

Licence types that are event-scoped by construction, the Special Event, Pop-Up, and Itinerant variants, are excluded as well. That removed 23,042 licences across 12 types, permits that typically expire within their first week. Those lives are short because the licence says so, and a lifespan that is intent rather than failure is the wrong thing to ask a survival model about.

Figure 1 shows Kaplan-Meier survival curves by `license_description`, the coarsest structure in the outcome before any model is fitted (estimator defined in section 4).



**Figure 1.** Kaplan-Meier survival curves by `license_description`. Groups beyond the seven most frequent are collapsed into (other) for this plot only. The vertical drop in the (other) curve at about two years, and the smaller steps at term multiples in other curves, are licence terms expiring. A business that does not renew exits at a term boundary, so endings cluster there.

## 3. Method

### 3.1 Model class

The target is a duration with incomplete observations, so the model is an accelerated failure time (AFT) model, XGBoost with its `survival:aft` objective. An observed ending enters as  $[t, t]$  and a censored row as  $[t, \text{infinity})$ , so every row contributes. The model predicts each row's median survival time in days, and a log-normal curve of fitted, shared width around that median gives the probability of surviving any horizon. A Cox proportional hazards baseline, the standard linear survival model, is fitted with lifelines' `CoxPHFitter` on the same features and answers whether the boosted model was necessary.

Numeric features pass through, missing values included (the Cox baseline gets train-window median imputation); text columns are one-hot encoded with the vocabulary refit per training window, so a fold's features reflect only what was on file by its split date.

### 3.2 Temporal validation and label re-censoring

Evaluation uses 5 expanding-window folds ordered by start date: the earliest 40% of rows is burn-in, and each fold trains on every row started before its split date and tests on the next block (sizes in Table 2).

Training labels are re-censored at each split date. A row started long before a split may have died after it, and its label contains that future, so every post-split death is rewritten as a censoring at the split. Omitting this raises scores by importing the future, which makes it the most consequential detail in the pipeline; it is a standalone function ( `cv.recensor` ) with dedicated tests.

### 3.3 Selection and calibration

Hyperparameters are selected once, on the first fold's training window, by an inner temporal split scored on held-out censored log-likelihood; concordance is scale-invariant and cannot see an unusable distribution width. The predictive scale, the reported curves' width, is calibrated on a temporal tail slice by a probe model that never trained on those rows; the selected loss scale was 1.2, the calibrated predictive scale 0.88.

The methodology is validated separately against synthetic data with known ground truth; the repository's synthetic report documents those checks.

## 4. Results

### 4.1 Discrimination

**Table 1.** Concordance index by model, pooled over 143,833 test rows with 95% percentile bootstrap intervals over 500 resamples. Higher is better; 0.500 is a coin flip. Fold-mean rows score each of the 5 folds separately and average. The interval resamples test rows with the fitted models held fixed, so it measures scoring precision, not stability across history; the fold spread is the guide to that.

| METHOD                               | CONCORDANCE (HARRELL'S C) | 95% INTERVAL   |
|--------------------------------------|---------------------------|----------------|
| XGBoost AFT (pooled)                 | 0.573                     | 0.571 to 0.575 |
| XGBoost AFT (fold mean)              | 0.585                     | not resampled  |
| Cox proportional hazards (fold mean) | 0.592                     | not resampled  |

Each fold refits both models on its own window, and the Cox risk scores are relative to that window, so fold-mean rows are the like-for-like comparison. The pooled row concatenates 5 separately fitted AFT models whose levels can differ, blurring cross-fold pairs; on this run it reads conservative next to the fold mean.

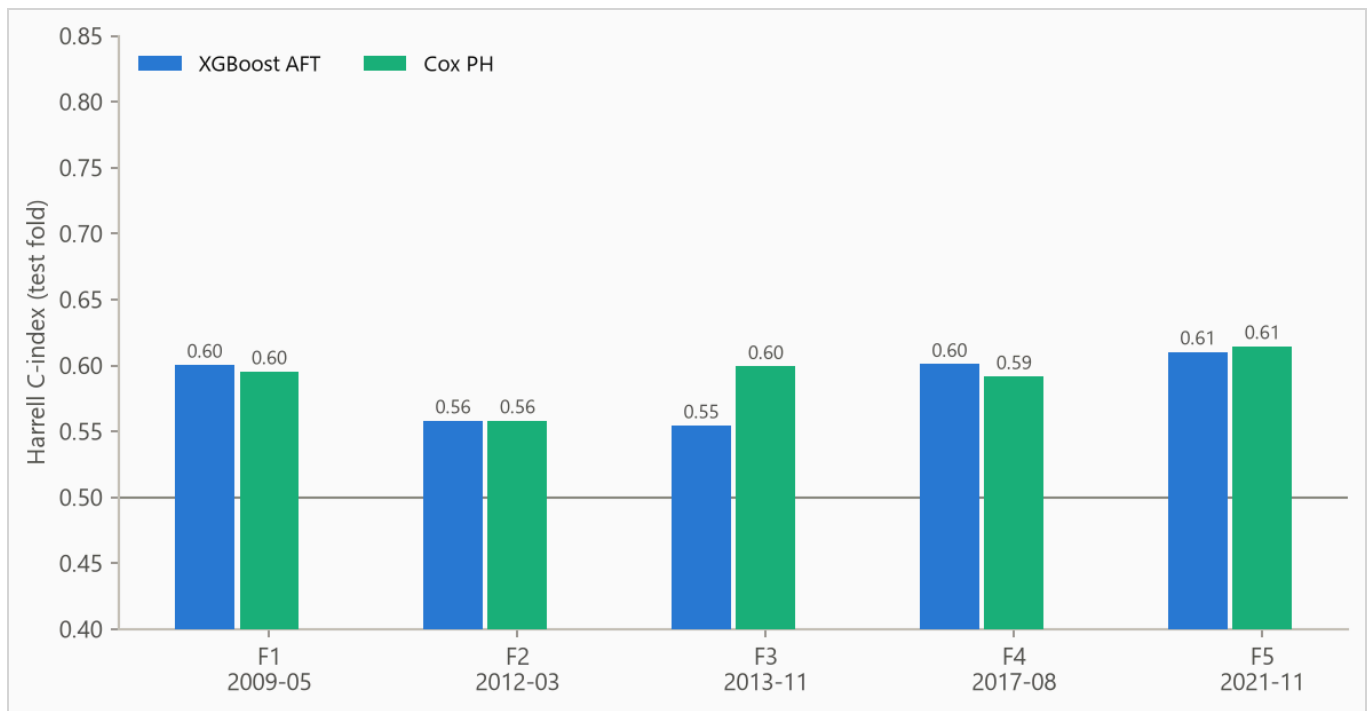
The weakest window is fold 3, starting 2013-11-27, at 0.554. That window's weakness has a measurable cause. Its test block sits across a shift in the city's category mix. The Home Occupation and Home Repair categories, carrying 11 percent of its training rows, stop appearing in the test block entirely. Regulated Business License nearly triples its share, and 2 percent of test rows are in categories the training window never saw. Re-selecting hyperparameters on this fold's own training window closes about a third of the gap to the Cox baseline. The remainder is consistent with the boosted model leaning harder than the linear baseline on category structure that had just moved.

The pooled concordance decomposes by `license_description` : group means alone score 0.613, and comparisons restricted to same-group rows score 0.510 (pair-weighted across 65 qualifying groups). The within-group distance above 0.500 is row-level skill; the rest is group membership.

Figure 2 plots Table 2' per-fold concordance.

**Table 2.** Per-fold results. Censoring is the share of each fold's test rows whose ending was not observed.

| FOLD | SPLIT DATE | TRAIN N | TEST N | CENSORED | AFT   | COX   |
|------|------------|---------|--------|----------|-------|-------|
| 1    | 2009-05-18 | 95,870  | 28,767 | 7%       | 0.600 | 0.595 |
| 2    | 2012-03-27 | 124,601 | 28,767 | 14%      | 0.558 | 0.558 |
| 3    | 2013-11-27 | 153,398 | 28,767 | 18%      | 0.554 | 0.599 |
| 4    | 2017-08-07 | 182,164 | 28,766 | 29%      | 0.601 | 0.592 |
| 5    | 2021-11-02 | 210,947 | 28,766 | 67%      | 0.610 | 0.615 |



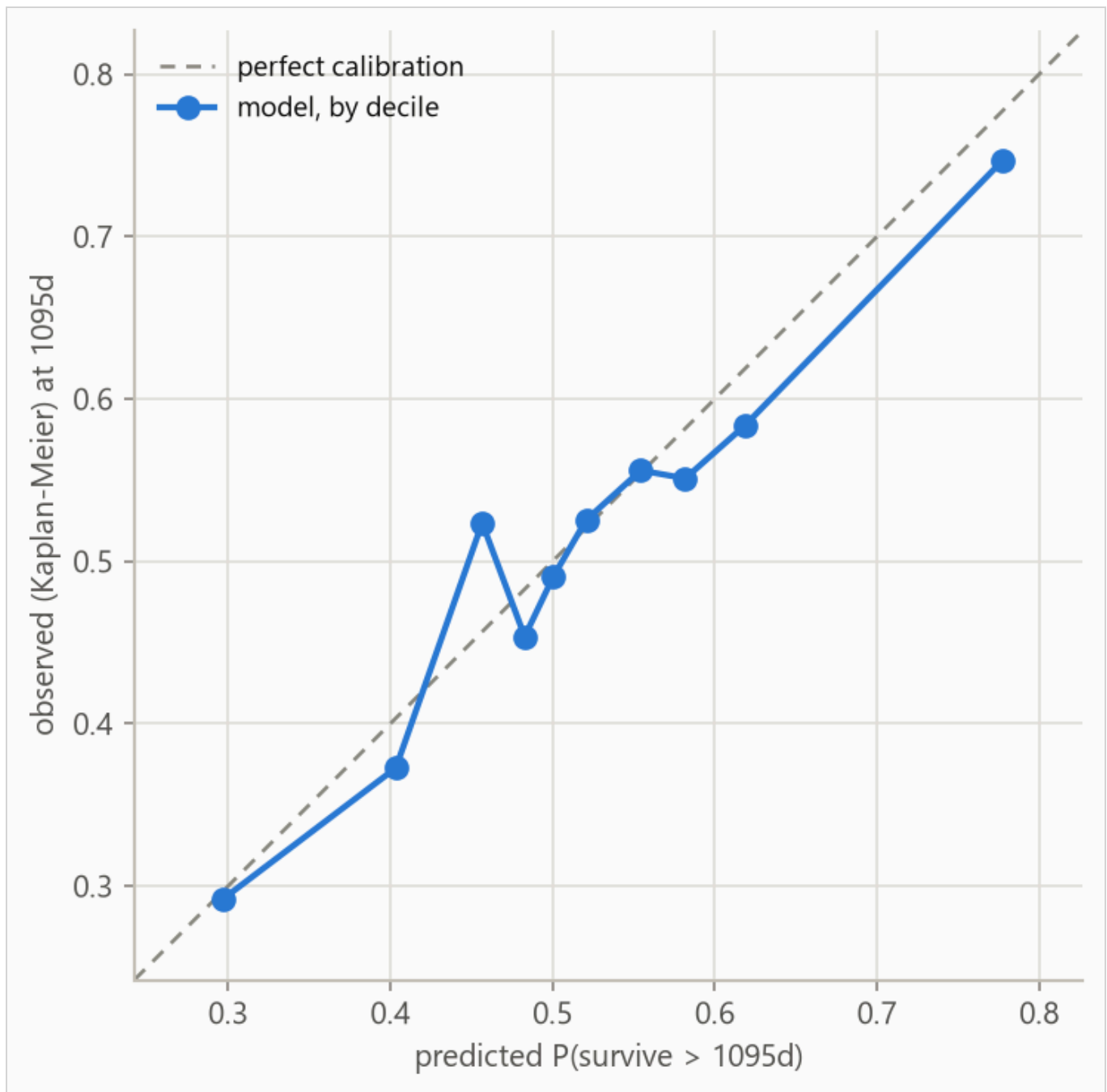
**Figure 2.** Concordance by fold, from 0.554 to 0.610. Folds differ in censoring mix, so cross-fold comparisons are indicative rather than exact.

## 4.2 Calibration

The Brier score, the mean squared error of a probability forecast (lower is better), is weighted by the inverse probability of censoring (IPCW) so censored rows do not bias it; the no-skill reference assigns every row the population's Kaplan-Meier marginal, the fraction surviving at each age with censored rows counted while observed.

**Table 3.** IPCW Brier score by horizon. Lower is better.

| HORIZON   | AFT   | COX   | NO-SKILL MARGINAL |
|-----------|-------|-------|-------------------|
| 365 days  | 0.097 | 0.088 | 0.081             |
| 1095 days | 0.239 | 0.231 | 0.250             |
| 1825 days | 0.217 | 0.212 | 0.227             |



**Figure 3.** Predicted against observed survival at 1095 days, by predicted decile. Observed frequencies are Kaplan-Meier estimates within each bin, so censored rows contribute correctly. The largest deviation is 0.067 in decile 3.

**Table 4.** Decile calibration at 1095 days, the values plotted in Figure 3. The observed value in each decile is a Kaplan-Meier estimate; when a decile's last observed row falls short of the 1095-day horizon the estimate carries its last value forward, so in small or heavily censored deciles the observed column is softer than its three decimals look.

| DECILE | N     | PREDICTED | OBSERVED (KM) | DEVIATION |
|--------|-------|-----------|---------------|-----------|
| 1      | 14492 | 0.297     | 0.292         | -0.005    |
| 2      | 14933 | 0.403     | 0.373         | -0.031    |
| 3      | 13805 | 0.457     | 0.524         | +0.067    |
| 4      | 15540 | 0.483     | 0.454         | -0.029    |
| 5      | 13533 | 0.501     | 0.491         | -0.010    |
| 6      | 14076 | 0.521     | 0.526         | +0.005    |
| 7      | 14318 | 0.554     | 0.556         | +0.002    |
| 8      | 15061 | 0.582     | 0.551         | -0.031    |
| 9      | 13874 | 0.619     | 0.584         | -0.035    |
| 10     | 14201 | 0.777     | 0.747         | -0.031    |

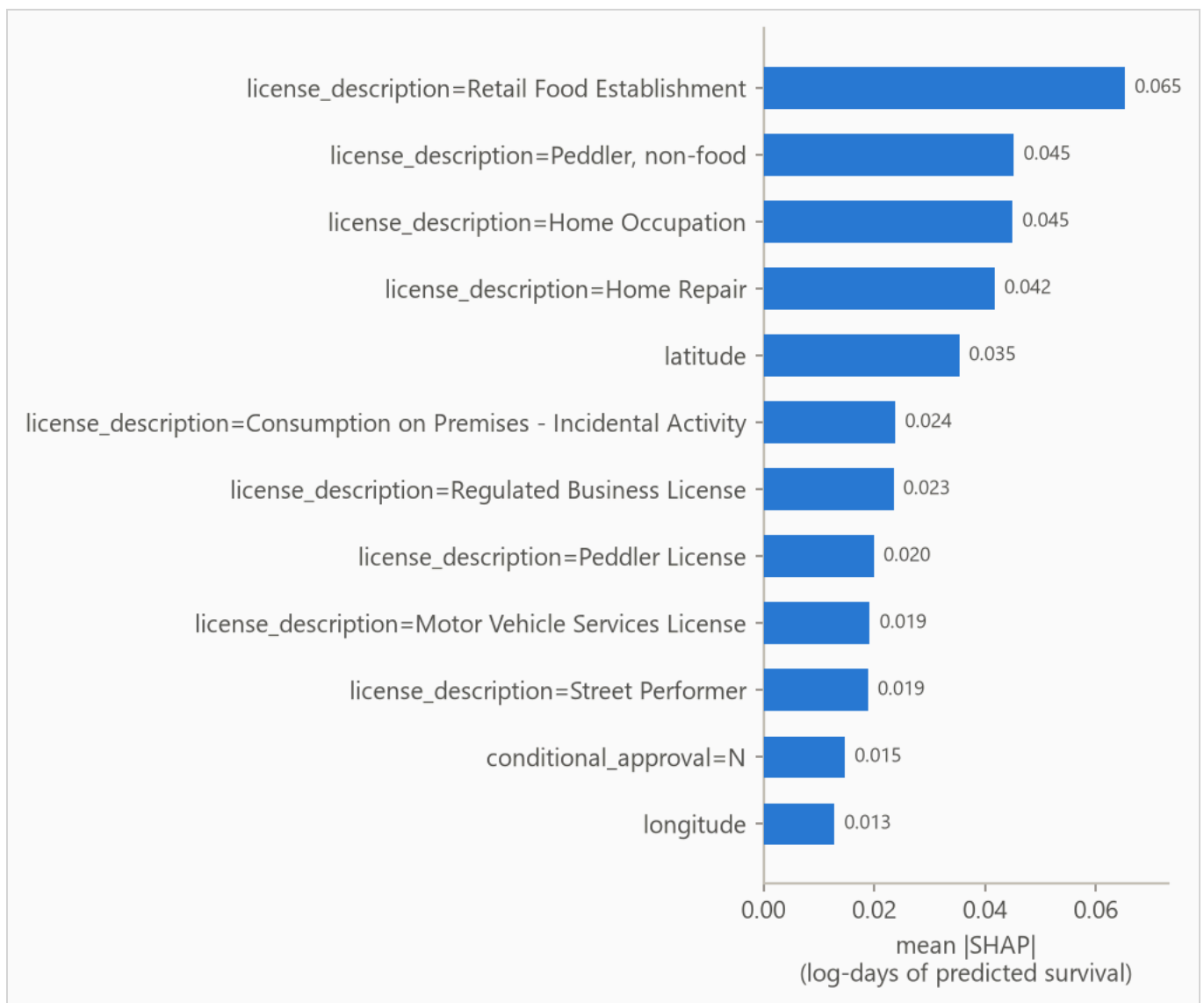
## 5. What the model uses

Feature attributions (SHAP values, for SHapley Additive exPlanations) are computed on the log scale of survival time; +0.3 multiplies predicted survival by about 1.35, and negative values shorten it.

Attributions are descriptive. Correlated features split credit by the model's internal choices as much as by the data, and the values are in-sample, computed on the final model's own training rows. Directions are more trustworthy than magnitudes.

Figure 4 ranks features, Table 5 lists the top eight, Figure 5 shows the per-row spread, and Figure 6 traces attribution against value.

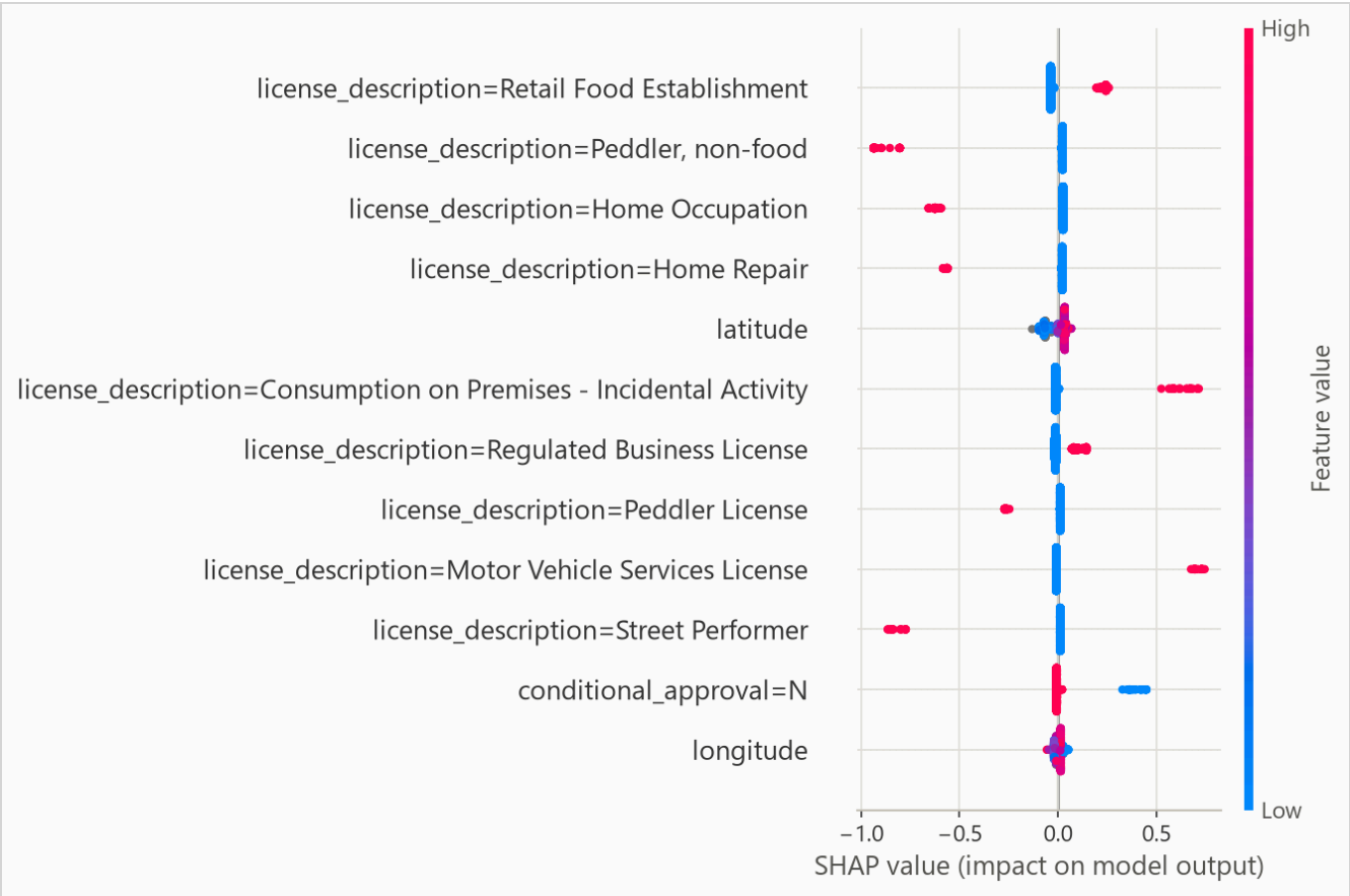




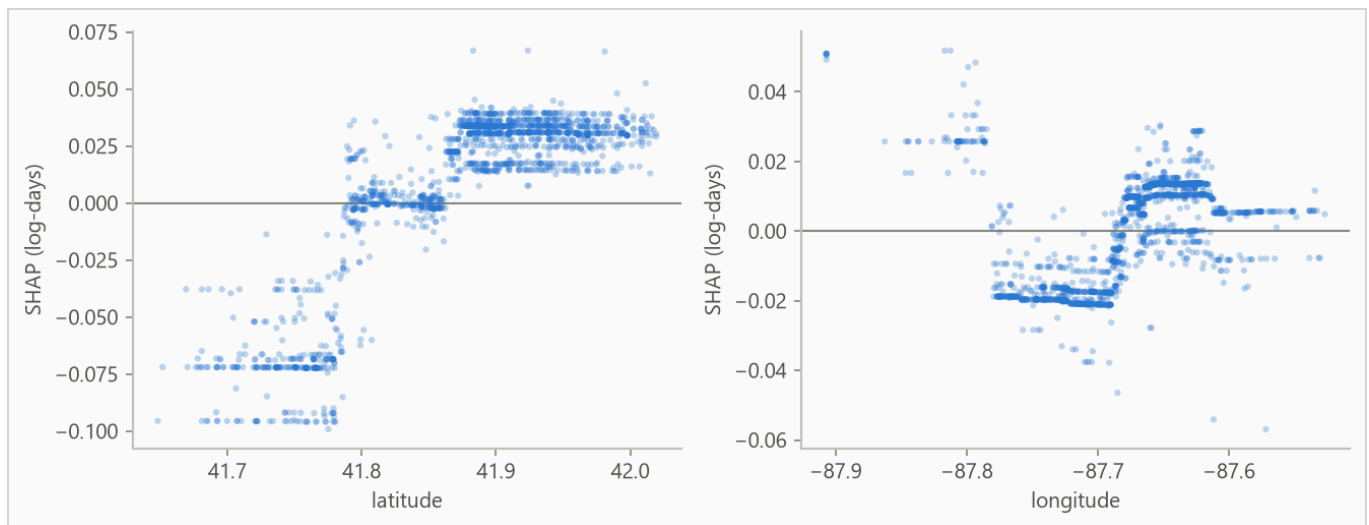
**Figure 4.** Mean absolute attribution by feature.

**Table 5.** Top eight features by mean absolute attribution.

| FEATURE                                                           | MEAN  ATTRIBUTION |
|-------------------------------------------------------------------|-------------------|
| license_description=Retail Food Establishment                     | 0.065             |
| license_description=Peddler, non-food                             | 0.045             |
| license_description=Home Occupation                               | 0.045             |
| license_description=Home Repair                                   | 0.042             |
| latitude                                                          | 0.035             |
| license_description=Consumption on Premises - Incidental Activity | 0.024             |
| license_description=Regulated Business License                    | 0.023             |
| license_description=Peddler License                               | 0.020             |



**Figure 5.** Per-row attributions across the explanation sample. For a yes/no feature, such as a one-hot category flag, the high (red) end simply means the row is in that category.



**Figure 6.** Attribution against feature value for the strongest numeric features; category flags carry no shape and stay in the ranking above. A run short on numeric features fills the grid with flags instead.

On this data there is no known mechanism to check the ranking against, so read it as "what this model leaned on", not as importance in the world.

## 6. Interpretation

The comparison worth acting on is the model choice. The Cox baseline edges the boosted model on the fold mean, 0.592 to 0.585, and the honest reading is that on this problem a penalized linear model is enough. Day-one paperwork mostly identifies risky licence categories: ranking rows by their category's mean prediction alone scores 0.613 against the model's pooled 0.573, and comparisons within a category score 0.510, barely above chance. The metadata says almost nothing about which handyman outlasts the others.

The one-year probabilities should not drive decisions. The model loses to the no-skill reference on Brier score at that horizon, 0.097 against 0.081, so treat it as an ordering tool at short horizons rather than a probability source there.

The printed bootstrap interval is narrower than the real uncertainty. Licences in the same category and the same stretch of time tend to fail together, and the interval resamples rows as if they were independent. The per-fold spread in the results section is the better guide to how the score moves across history.

## 7. Limitations

---

**Absolute probabilities are not usable at 365 days.** The model loses to the no-skill forecast on the censoring-weighted Brier score there. Ranking and probability quality are separate, so use this model to order rows, not to act on absolute probabilities at those horizons.

**Censoring may be informative.** The evaluation assumes rows stop being observed for reasons unrelated to their risk; early exits of rows about to end anyway bias survival estimates upward.

**Endings near the cutoff are provisional.** Where no cancellation exists, a licence's end is its latest recorded expiry, so a licence whose term ran out shortly before the cutoff but that renews late is recorded as an observed closure. Deaths dated close to the cutoff therefore include some licences that will turn out to have survived; how many is not measured here.

**Every row shares one curve shape.** The predictive scale is a single fitted number, so the model shifts a row's survival curve earlier or later but never reshapes it; per-row width is future work.

**Harrell's concordance is biased under heavy censoring,** and the final fold is 67% censored; read Table 2 with that in mind.

## 8. Reproducing this run

---

Python 3.11 or later. From the repository root:

```
python scripts/run_fit_evaluate.py --data datasets/chicago_licences.csv.gz --name chicago_licences
python scripts/run_build_report.py --run reports/chicago_demo
```

Every number is read from the run's `metrics.json` at build time, and a figure the prose never cites fails the build. `requirements.txt` pins the exact versions the committed numbers came from.

---

Generated from `reports/chicago_demo/metrics.json` by `scripts/run_build_report.py --run`. 239,721 rows, 143,833 out-of-time test rows.